

RecurShield

Recurring Bill & Subscription Management System

Prepared by:

Abdul Razak Mansaray

[GitHub Repo](#)

Chapter 1: Vision and High-Level Architecture

1.1 Executive Vision: The "What" and the "Why"

Managing everyday personal finances has become increasingly fragmented. Most people no longer deal with just a single monthly utility bill; instead, they juggle dozens of recurring micro-charges across mobile data plans, streaming services, software subscriptions, insurance policies, and automated cloud backups. Because these charges hit on different calendar days across multiple credit cards and bank accounts, tracking total commitments becomes nearly impossible. As a result, consumers face chronic financial leakage—consistently paying for forgotten trials, quiet price hikes, and services they stopped using long ago.

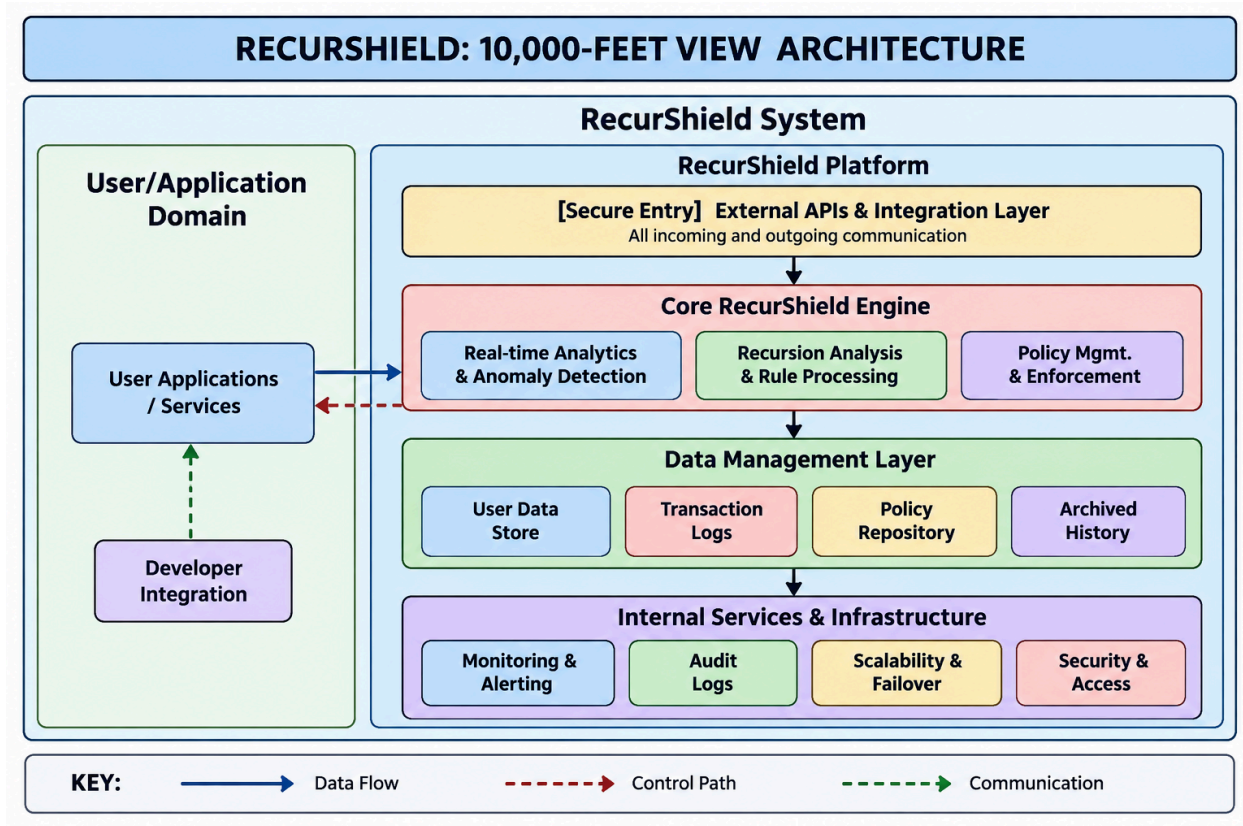
RecurShield is a cloud-native platform designed to eliminate this subscription blindness. The system aggregates a user's recurring expenses into a single control panel, detects billing cycles, and evaluates whether a service remains worthwhile based on actual user engagement. Rather than merely presenting numbers on a graph, RecurShield flags redundant tools, alerts users to surprise price jumps, and provides actionable recommendations on what to cancel, keep, or downgrade.

Building this as a cloud-native application on distributed infrastructure is an intentional design choice rather than an afterthought:

- **Failure Isolation:** Ingesting financial data requires handling unpredictable external network calls. If the bank-sync logic fails or times out, the user dashboard and active notification systems stay online without interruption.
- **Bursty Workload Scaling:** Bill analysis runs heaviest around common billing windows (like the first or last day of the month). A containerized setup lets us scale the calculation pods on demand without wasting compute power during quiet hours.
- **Separation of Sensitive State:** User authentication and ledger records can be locked behind private cluster networking, keeping them completely hidden from public access points.

1.2 10,000-Foot Architectural Blueprint

The system uses a classic decoupled three-tier cloud pattern, separating public entry, stateless application compute, and stateful storage.



1.3 Architecture Walkthrough

External traffic hits the system through a centralized Ingress Controller, which handles TLS termination and inspects incoming session tokens before allowing traffic into the cluster. This keeps individual backend containers free from managing SSL certificates directly.

Once verified, requests travel across internal private networks (**ClusterIP**) to either the NGINX frontend or the Core API Service. The Core API handles the business logic: registering accounts, parsing recurring transaction intervals, and tracking billing dates. Heavy analytical work—such as scanning for duplicate subscriptions or scoring whether an account is underutilized—is offloaded to a dedicated Recommendation Engine Pod.

Because containers are inherently ephemeral and can restart at any point, long-term application state never lives inside the container filesystems. Transaction records and user profiles are stored in a PostgreSQL database backed by a persistent storage claim, while frequently accessed subscription metrics sit in a Redis caching pod to keep the dashboard responsive without slamming the database with repetitive queries.

Chapter 2: Implementation Roadmap & Cloud Strategy

Translating the RecurShield design into working software requires building modular, containerized components that can run on an orchestrator like Kubernetes on bare-metal testbeds (such as CloudLab). The implementation focuses on container design, cluster networking, storage persistence, and automated delivery.

2.1 Containerizing the Microservices (Docker)

Each tier runs as a specialized container built with **multi-stage Dockerfiles** to keep final image sizes small and remove unnecessary build tools from production:

- **Frontend Web Client:** Built with React/TypeScript, compiled into static assets, and served via an Alpine-based NGINX container. This keeps the delivery footprint under 25 megabytes.
- **Core API Pod:** Implemented in Python using FastAPI. A multi-stage build installs Python dependencies in a builder stage and copies only the installed libraries into a clean, minimal runtime container.
- **Cross-Architecture Builds:** Development happens locally on an Apple M2 laptop running ARM64 hardware, while testing nodes on CloudLab run standard x86_64/AMD64 processors. To avoid deployment failures, images will be compiled for both architectures using `docker buildx` and tagged in a central registry before deployment.

2.2 Orchestration and Cluster Networking (Kubernetes)

Workloads will run inside a custom Kubernetes namespace (e.g., `recurshield-prod`) to keep resources organized and prevent naming collisions:

- **Deployments:** The API and frontend run as Kubernetes `Deployments`, allowing the cluster to maintain replica counts, handle zero-downtime rolling updates, and self-heal by automatically restarting crashed containers.
- **Internal Routing:** Pods communicate internally using Kubernetes `ClusterIP` services, which use internal DNS names (like `core-api-service.recurshield-prod.svc.cluster.local`) rather than hardcoded IP addresses. External access during class testing will be handled through an Ingress controller or a dedicated `NodePort` mapping.
- **Decoupling Configuration:** Plain configuration details (database hostnames, ports, logging levels) are passed into containers using Kubernetes `ConfigMaps`, while sensitive values (database credentials, JWT signing secrets) are mounted using Kubernetes `Secrets`. Nothing secret is ever hardcoded into Dockerfiles or source files.

2.3 Storage Architecture and Persistence

Because billing and subscription records must survive pod crashes, node reboots, and redeployments, data storage is separated from compute:

- **Persistent Volumes (PV & PVC):** The PostgreSQL database will mount persistent storage through a Kubernetes `PersistentVolumeClaim` backed by an NFS server or CloudLab storage partition. If a database pod dies, Kubernetes brings up a replacement pod on any healthy node and reattaches the same volume claim.
- **Redis Caching Tier:** A standalone Redis pod handles fast session lookups and stores calculated recommendation summaries, preventing repetitive aggregation queries from hitting disk storage.

2.4 CI/CD Automation (GitHub Actions)

Deployments will be managed through a clear continuous integration workflow:

- Every pull request to the `main` branch on GitHub triggers a GitHub Actions pipeline.
- The pipeline checks the code for syntax issues and runs automated unit tests.
- If tests pass, GitHub Actions uses `docker buildx` to build multi-platform container images (`linux/amd64` and `linux/arm64`) and pushes the resulting artifacts to Docker Hub or the GitHub Container Registry with a specific version tag.
- Versioned Kubernetes manifest files in the repository ensure that any member of the team—or the course instructor—can reproduce the full cluster environment on demand.

Abdul Razak Mansaray

Philadelphia, PA | +13124878093 | abdulrazakmans1@gmail.com

Professional Summary

Computer Science graduate student with experience in AI, data science, cloud computing, and IT systems. Skilled in Python, Pandas, SQL, Docker, Kubernetes, Linux, and Git, with ICT leadership and teaching experience. Currently developing an **agentic AI research assistant** and expanding practical data analysis skills.

Experience

West Chester University of Pennsylvania 2026 - Present

Independent Research - Agentic AI / Data Science

- Developing Zaqvion, a human-in-the-loop AI Research Intelligence Agent
- Exploring human-AI interaction, responsible AI, agentic workflows, transparency, provenance, and researcher control as core design principles for AI-assisted research.

NCTVA

April 2019- Aug 2023

Head of ICT Department

- Managed internal ICT trainings
- Led the implementation of the Integrated Certification Software
- Expanded the NCTVA's website to a TVET communication platform
- Administered routine maintenance for the ICS Servers at the Zoodlab server housing
- Designed the internal Certification System and collaborated in TVET curriculum design
- Led the internal quality assurance committee on certificate validation

Luban College of Social Sciences and Technology

Sep 2017- March 2022

College Lecturer- Computer Science

- Designed course materials for the students within the computer science department
- Educated and challenged students through comprehensive lectures

Education

West Chester University of Pennsylvania- West Chester, PA

Dec 2026

MS in Computer Science

Eastern Illinois University- Charleston, IL

May 2025

MS in Cyber Security (transferred)

University of Sierra Leone- Freetown, Sierra Leone

Feb 2019

B.Sc. in Information Systems

Core Competencies

- AI / Agentic AI: LLM-based agents, human-in-the-loop AI, research intelligence systems
- Data Science & Analytics: Python, Pandas, data cleaning, exploratory data analysis, SQL
- Programming & Databases: Python, MySQL
- Cloud / DevOps: Google Cloud, Docker, Kubernetes, Git, Linux, Bash, PowerShell